CS445 - Basics of Web Programming

Lab Assignment 6 - NOTES

Event Board: JavaScript basics, the DOM, fetch and form validation

Team members

Role	Roll number	Name
Team Leader	20261651014	Ankit Sahoo
Member 1	20261651095	Vikas Gupta
Member 2	20261651064	Prakash Rai
Member 3	20261651070	Ram Ranjan
Member 4	20261651090	Sukant Sharma

Files submitted: index.html, style.css, script.js, events.json, NOTES.pdf

Note: the page loads events.json with fetch(), so open it through a local server (for example the VS Code *Live Server* extension or `python -m http.server`) and not by double-clicking the file; browsers block fetch() on file:// URLs.

1. let vs const - when to use each, and why avoid var?

I use `const` by default, for any binding that is never reassigned (for example `SITE_NAME`, `LOW_SEATS_LIMIT`, `DATA_URL` and every DOM element). I use `let` only when the value really changes: `isLoading`, `searchTerm` and `allEvents`. Note that `const` stops *reassignment*, not mutation; I can still push or edit objects inside a const array. I avoid `var` because it is function-scoped (it ignores block braces), is hoisted and initialised as `undefined`, and can be re-declared silently; all of these cause hard-to-find bugs. `let` and `const` are block-scoped and cannot be used before their declaration.

2. Data types used, with one value of each

Type	Example value in script.js
string	`SITE_NAME = "Event Board - IIIT Vadodara"`
number	`LOW_SEATS_LIMIT = 5 (also seats: 20)`
boolean	`isLoading = true (also online: false)`
array	`allEvents = [ {...}, {...}, {...}, {...} ]`
object	`{ id: 1, title: "Coding Event", seats: 20, ... }`
null / undefined	`undefined: ev found with .find() when no id matches`

3. One event object, and why an array of objects is a good model

```
{ "id": 2, "title": "E-Sports", "date": "2026-11-10",
  "category": "Gaming", "seats": 3, "online": true }
```

Each event is one object that groups its own properties (text, number, boolean), and the array keeps them in order and lets me loop over them. The array is the single source of truth: the cards, the table, the search filter and the seat summary are all produced from the same data, so I change the data once and the whole page stays consistent. Adding a new event means adding one object in events.json, with no HTML edits. It also matches the JSON format that fetch() returns.

4. Control-flow rule (A4) and the labels produced

```
function getAvailability(seats) {
  if (seats === 0) {
    return "Full";
  } else if (seats < LOW_SEATS_LIMIT) {
    return "Few left";
  } else {
    return "Open";
  }
}
```

Labels produced: **Coding Event** (20 seats) = Open; **E-Sports** (3 seats) = Few left; **Music** (0 seats) = Full; **Dance** (12 seats) = Open.

5. One function (A6) and one arrow function with an array method (A7)

```
function formatEvent(ev) {
  const label = getAvailability(ev.seats);
  const mode = ev.online ? "Online" : "On campus";
  return `${ev.title} - ${formatDate(ev.date)} - ${label} (${mode})`;
}

const getOpenEvents = (events) => events.filter(ev => ev.seats > 0);
```

`formatEvent` returns a readable string; for the first event it returns `Coding Event - Monday, 9 Nov 2026 - Open (On campus)`. I use it as each card's aria-label and in the console. `getOpenEvents` returns a **new array** containing only events whose seats are greater than 0 (3 events: Coding, E-Sports, Dance); the original array is not changed. It feeds the summary line "3 of 4 events still have seats." I also use `map` in `getTitles`, which returns `["Coding Event", "E-Sports", "Music", "Dance"]`.

Scope (A8): a `const` declared inside a function (such as `label` in `formatEvent`) is block/function scoped: the variable exists only between that function's braces, and the engine creates a new one on every call. Code outside cannot see it, so using it there throws a ReferenceError. This keeps variables small, prevents name clashes, and lets the memory be freed after the function returns.

6. Elements selected, methods used, and why prefer querySelector

With `document.querySelector`: `.menu-toggle`, `.nav-links`, `#event-table tbody`, `#status`, `#search`, `#event-form`, `#person-name` and `#form-message`. With `document.getElementById`: `event-cards`. With `document.querySelectorAll`: `#event-form input, #event-form textarea` (the form fields). I prefer `querySelector` because it accepts any CSS selector (id, class, attribute, descendant), so I use one syntax I already know from my CSS; it returns the first match or null; and it is easy to change when the markup changes. `getElementById` is only slightly faster and works for ids only.

7. Event bubbling, using event.target (B5)

When something is clicked, the event runs on that element first and then **bubbles up** through its parents. I attach **one** click listener to the container `#event-cards` instead of one per button. In the handler, `event.target` is the exact element clicked (for example the RSVP button inside one card), while `event.currentTarget` is the container that owns the listener. I check `event.target.classList.contains("rsvp")` and read `dataset.id` to find which event was clicked. This also works for cards that are created later by JavaScript.

8. The three fetch states and what the user sees

```
async function loadEvents() {
  setStatus("Loading events...", "loading");
  try {
    const response = await fetch(DATA_URL);
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    allEvents = await response.json();
    applyFilter(); updateSummary();
  } catch (error) {
    setStatus("Sorry, we could not load ...", "error");
  }
}
```

Loading: an orange-edged box reads "Loading events..." while the request is in progress. **Success:** the box turns green with "3 of 4 events still have seats.", and the cards and table are built from events.json. **Failure:** (file missing, network error or invalid JSON) a red bold message says the events could not be loaded, and the details are written to the console with console.error. Because fetch() only rejects on network errors, I also check response.ok and throw my own error for HTTP errors such as 404.

9. Validation: built-in attributes, custom rule, and why the server must re-check

```
<input type="text"  id="person-name" required>
<input type="date"  id="event-date"  required>
```

```
<input type="email" id="email"       required>
<input type="text"  id="location"    required
       pattern="[A-Za-z0-9 ,.\-]{3,60}">
<textarea id="description" maxlength="200"></textarea>

function checkName() {
  if (nameInput.value.trim().length < 2) {
    nameInput.setCustomValidity("Name must be at least 2 characters long.");
  } else {
    nameInput.setCustomValidity("");
  }
}
form.addEventListener("submit", (event) => {
  checkName();
  if (!form.checkValidity()) {
    event.preventDefault();
    const firstInvalid = form.querySelector(":invalid");
    formMessage.textContent = firstInvalid.validationMessage;
    firstInvalid.focus();
  }
});
```

The form has **novalidate** so the browser's own pop-ups do not appear; instead, the first invalid field's validationMessage is shown in the red message line, the field gets a red border, and focus moves to it. After the error is fixed and the form is submitted again, a green thank-you message appears. **The server must validate again** because client-side checks only improve the user experience: anyone can disable JavaScript, edit the HTML in DevTools, or send a request directly (with curl or a script) that never touches my form. The server is the only place that can be trusted to enforce rules, protect the database, and reject malicious or malformed data.

10. Final script.js

Live Server link (local, runs on the team's computer):
http://127.0.0.1:5501/NewCampusEvents.html/index.html

The complete final script.js is attached in the appendix on the following pages.

Appendix - final script.js

```
const SITE_NAME = "Event Board - IIIT Vadodara";
const LOW_SEATS_LIMIT = 5;
const DATA_URL = "events.json";
let isLoading = true;
let searchTerm = "";

let allEvents = [];

const menuToggle   = document.querySelector(".menu-toggle");
const navLinks     = document.querySelector(".nav-links");
const cardsBox     = document.getElementById("event-cards");
const tableBody    = document.querySelector("#event-table tbody");
const statusBox    = document.querySelector("#status");
const searchBox    = document.querySelector("#search");
const form         = document.querySelector("#event-form");
const nameInput    = document.querySelector("#person-name");
const formMessage  = document.querySelector("#form-message");
const formFields   = document.querySelectorAll("#event-form input, #event-form textarea");

console.log(`${SITE_NAME} - script loaded`);
console.log("Form fields found:", formFields.length);

function getAvailability(seats) {
  if (seats === 0) {
    return "Full";
  } else if (seats < LOW_SEATS_LIMIT) {
    return "Few left";
  } else {
    return "Open";
  }
}

function formatDate(isoDate) {
  const d = new Date(`${isoDate}T00:00:00`);
  const weekday = d.toLocaleDateString("en-US", { weekday: "long" });
  const rest = d.toLocaleDateString("en-GB", { day: "numeric", month: "short", year: "numeric" });
  return `${weekday}, ${rest}`;
}

function formatEvent(ev) {
  const label = getAvailability(ev.seats);
  const mode = ev.online ? "Online" : "On campus";
  return `${ev.title} - ${formatDate(ev.date)} - ${label} (${mode})`;
}

const getOpenEvents = (events) => events.filter(ev => ev.seats > 0);
const getTitles = (events) => events.map(ev => ev.title);

function setStatus(message, type) {
  statusBox.textContent = message;
  statusBox.className = `status ${type}`;
  statusBox.hidden = message === "";
}

function updateSummary() {
  const openCount = getOpenEvents(allEvents).length;
  setStatus(`${openCount} of ${allEvents.length} events still have seats.`, "success");
}

function renderEvents(list) {
  cardsBox.textContent = "";
  tableBody.textContent = "";

  if (list.length === 0) {
    const empty = document.createElement("p");
    empty.classList.add("empty-message");
    empty.textContent = "No events match your search.";
    cardsBox.appendChild(empty);
    return;
  }

  for (const ev of list) {
    const label = getAvailability(ev.seats);
    const slug = label.toLowerCase().replace(" ", "-");

    const card = document.createElement("article");
    card.classList.add("card");
    card.setAttribute("aria-label", formatEvent(ev));
    card.dataset.id = ev.id;
    const title = document.createElement("h3");
```

```
    title.textContent = ev.title;

    const day = document.createElement("p");
    day.textContent = `Day: ${formatDate(ev.date)}`;

    const badge = document.createElement("span");
    badge.classList.add("badge");
    badge.textContent = ev.category;

    const mode = document.createElement("p");
    mode.textContent = ev.online ? "Online event" : "On campus";

    const availability = document.createElement("p");
    availability.classList.add("availability", `avail-${slug}`);
    availability.textContent = `${label} - ${ev.seats} seats`;

    const rsvp = document.createElement("button");
    rsvp.type = "button";
    rsvp.classList.add("rsvp");
    rsvp.dataset.id = ev.id;
    rsvp.textContent = "RSVP";

    if (ev.seats === 0) {
      card.classList.add("full");
      rsvp.textContent = "Full";
      rsvp.disabled = true;
      rsvp.style.cursor = "not-allowed";
    }

    card.append(title, day, badge, mode, availability, rsvp);
    cardsBox.appendChild(card);

    const row = document.createElement("tr");
    for (const text of [ev.title, formatDate(ev.date), String(ev.seats), label]) {
      const cell = document.createElement("td");
      cell.textContent = text;
      row.appendChild(cell);
    }
    tableBody.appendChild(row);
  }
}

function applyFilter() {
  const term = searchTerm.trim().toLowerCase();
  const visible = allEvents.filter(ev =>
    ev.title.toLowerCase().includes(term) ||
    ev.category.toLowerCase().includes(term)
  );
  renderEvents(visible);
}

searchBox.addEventListener("input", (event) => {
  searchTerm = event.target.value;
  applyFilter();
});

searchBox.addEventListener("keydown", (event) => {
  if (event.key === "Escape") {
    searchBox.value = "";
    searchTerm = "";
    applyFilter();
  }
});

menuToggle.addEventListener("click", () => {
  const isOpen = navLinks.classList.toggle("open");
  menuToggle.setAttribute("aria-expanded", String(isOpen));
});

cardsBox.addEventListener("click", (event) => {
  const clicked = event.target;
  console.log("clicked:", clicked, "| listener is on:", event.currentTarget);

  if (!clicked.classList.contains("rsvp")) return;

  const ev = allEvents.find(item => item.id === Number(clicked.dataset.id));
  if (ev && ev.seats > 0) {
    ev.seats -= 1;
    console.log(`RSVP for "${ev.title}" - seats left: ${ev.seats}`);
    applyFilter();
    updateSummary();
  }
});
async function loadEvents() {
```

```
  isLoading = true;
  setStatus("Loading events...", "loading");
  try {
    const response = await fetch(DATA_URL);
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }
    allEvents = await response.json();
    console.log("Loaded events:", allEvents);
    console.log("Titles:", getTitles(allEvents));
    applyFilter();
    updateSummary();
  } catch (error) {
    console.error("Could not load events:", error);
    setStatus("Sorry, we could not load the events right now. Please try again later.", "error");
  } finally {
    isLoading = false;
  }
}

function checkName() {
  if (nameInput.value.trim().length < 2) {
    nameInput.setCustomValidity("Name must be at least 2 characters long.");
  } else {
    nameInput.setCustomValidity("");
  }
}

nameInput.addEventListener("input", checkName);

form.addEventListener("submit", (event) => {
  checkName();
  form.classList.add("was-submitted");

  if (!form.checkValidity()) {
    event.preventDefault();
    const firstInvalid = form.querySelector(":invalid");
    formMessage.textContent = firstInvalid.validationMessage;
    formMessage.className = "form-message error";
    firstInvalid.focus();
    return;
  }

  event.preventDefault();
  formMessage.textContent = `Thanks, ${nameInput.value.trim()}! Your event was submitted.`;
  formMessage.className = "form-message success";
  form.reset();
  form.classList.remove("was-submitted");
});

loadEvents();
```